

GNU Make for Reproducible Research

Automating and Documenting Data Analysis Pipelines

UBC BIOF Statistics Workshop

Objective

Today's Goal:

Learn how to use GNU Make to automate your research workflows

What you'll learn:

- Why Make > Shell scripts for pipelines
- Basic Make syntax
- Real data analysis example

The Problem: Manual Workflows 🤔

Typical research workflow:

```
# Download data
Rscript download_data.R

# Clean data
Rscript clean_data.R

# Analyze
Rscript analyze.R

# Make figures
Rscript make_figures.R
```

Problems:

The Problem Continued

What happens when you update the data?

-  Re-run everything (slow!)
 -  Manually track what needs updating (error-prone!)
 -  Forget to update downstream files (wrong results!)
-  **Solution: GNU Make automatically tracks dependencies!**

What is GNU Make?

“GNU is not unix”, a build automation tool that:

1. Tracks dependencies between files
2. **Runs only what's needed** when files change
3. Documents your workflow in one place
4. **Ensures reproducibility**

Originally designed for compiling software, but perfect for data analysis!

Basic Makefile Syntax

The tool itself is a binary called `make` that runs with `Makefile`.

```
target: dependencies
      command
```

Example:

```
1 clean_data.csv: raw_data.csv
2     Rscript clean_data.R
```

- `clean_data.csv` = **target** (what you want to create)
- `raw_data.csv` = **dependency** (what you need)
- `Rscript clean_data.R` = **command** (how to make it)

 **Important: Commands MUST start with a TAB, not spaces!**

How Make Works

When you run `make clean_data.csv`:

1. Check if `clean_data.csv` exists
2. Check if `raw_data.csv` is newer than `clean_data.csv`
3. If yes → run the command
4. If no → skip (already up-to-date!)

This is the magic! 🎩 ✨

Make only rebuilds what's necessary based on file timestamps.

Make vs Shell Scripts

Makefile approach:

```
results.csv: raw_data.csv clean.R
    Rscript clean.R

plots/figure1.png: results.csv plot.R
    Rscript plot.R
```

Benefits:

- Only re-runs when inputs change
- Automatic dependency resolution
- Self-documenting workflow
- Can run independent tasks in parallel

Make vs Shell Scripts

Shell script approach:

```
#!/bin/bash
# run_all.sh
Rscript download.R
Rscript clean.R
Rscript analyze.R
Rscript plot.R
```

Limitations:

- Runs everything every time
- No dependency tracking
- Manual ordering
- No parallelization

Real Example: Data Analysis Pipeline

Let's build a complete analysis pipeline step by step!

Our pipeline:

1. Download data from the web
2. Clean and preprocess
3. Run statistical analysis
4. Generate figures
5. Create final report

Step 1: Download Data

Create `Makefile`:

```
data/raw_data.csv:  
    mkdir -p data  
    wget -O data/raw_data.csv \  
        https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
```

Run it:

```
make data/raw_data.csv
```

Run again:

```
make data/raw_data.csv  
# make: 'data/raw_data.csv' is up to date.
```

Step 2: Clean Data

Add to **Makefile**:

```
data/clean_data.csv: data/raw_data.csv scripts/clean.R  
    Rscript scripts/clean.R
```

What this means:

- To create `data/clean_data.csv`...
- You need `data/raw_data.csv` AND `scripts/clean.R`
- Run the R script to process it

If you edit `clean.R`, Make knows to re-run this step!

Step 3: Statistical Analysis

```
results/statistics.txt: data/clean_data.csv scripts/analyze.R  
  mkdir -p results  
  Rscript scripts/analyze.R
```

Chain of dependencies:

`raw_data.csv` → `clean_data.csv` → `statistics.txt`

Make automatically handles the chain!

Step 4: Generate Figures

```
figures/plot1.png: results/statistics.txt scripts/plot.R  
  mkdir -p figures  
  Rscript scripts/plot.R
```

Now we have a full pipeline:

raw_data.csv → clean_data.csv → statistics.txt → plot1.png

Complete Makefile Example

```
# Download data
data/raw_data.csv:
    mkdir -p data
    wget -O data/raw_data.csv https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv

# Clean data
data/clean_data.csv: data/raw_data.csv scripts/clean.R
    Rscript scripts/clean.R

# Analyze
results/statistics.txt: data/clean_data.csv scripts/analyze.R
    mkdir -p results
    Rscript scripts/analyze.R

# Plot
figures/plot1.png: results/statistics.txt scripts/plot.R
    mkdir -p figures
    Rscript scripts/plot.R
```

Running Your Pipeline

Run the entire pipeline:

```
make figures/plot1.png
```

Make will:

1. Check if `data/raw_data.csv` exists → download if needed
2. Check if `data/clean_data.csv` is up-to-date → clean if needed
3. Check if `results/statistics.txt` is up-to-date → analyze if needed
4. Check if `figures/plot1.png` is up-to-date → plot if needed

Only runs what's necessary!

Phony Targets

Some targets don't create files:

```
.PHONY: clean all

all: figures/plot1.png

clean:
    rm -f data/clean_data.csv
    rm -f results/statistics.txt
    rm -f figures/plot1.png
```

Usage:

```
make all    # Build everything
make clean  # Remove generated files
```

Sometimes convenience target

Sometimes, you just want to have everything to send out to your collaborators

```
collaboration-packet.zip: data-wrangling.R raw-data.csv paper.Rmd comments.txt  
Rscript data-wrangling.R clean-data.csv  
gzip collaboration-packet.zip clean-data.csv paper.Rmd comments.txt
```

Variables for Cleaner Makefiles

```
# Define variables
R = Rscript
DATA_DIR = data
RESULTS_DIR = results
FIGURES_DIR = figures

# Use variables
$(DATA_DIR)/clean_data.csv: $(DATA_DIR)/raw_data.csv scripts/clean.R
    $(R) scripts/clean.R

$(FIGURES_DIR)/plot1.png: $(RESULTS_DIR)/statistics.txt scripts/plot.R
    mkdir -p $(FIGURES_DIR)
    $(R) scripts/plot.R
```

Benefits: Easy to change paths or commands in one place!

Pattern Rules

For repetitive tasks:

```
# Convert all .csv files to .xlsx  
%.xlsx: %.csv  
    python convert_to_excel.py $< $@
```

Special variables:

- $\$<$ = first dependency (%.csv)
- $\$@$ = target (%.xlsx)

This rule works for ANY csv $\rightarrow$ xlsx conversion!

Real-World Example: Complete Pipeline

```
.PHONY: all clean

# Variables
R = Rscript
DATA = data/iris.csv
CLEAN_DATA = data/iris_clean.csv
RESULTS = results/model_summary.txt
FIGURE = figures/iris_plot.png

# Default target
all: $(FIGURE)

# Pipeline
$(DATA):
    mkdir -p data
    wget -O $(DATA) \
        https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
```

Parallel Execution

Make can run independent tasks simultaneously:

```
all: figure1.png figure2.png figure3.png
```

```
figure1.png: data.csv script1.R
```

```
    Rscript script1.R
```

```
figure2.png: data.csv script2.R
```

```
    Rscript script2.R
```

```
figure3.png: data.csv script3.R
```

```
    Rscript script3.R
```

Run in parallel:

```
make -j 3 all # Run 3 jobs simultaneously
```

When to Use Make vs R Scripts

Use Make when:

- Multiple steps in your pipeline
- Different tools (R, Python, bash)
- Long-running computations
- Need to track dependencies

Stick with R scripts when:

Best Practices

1. **One Makefile per project** in the root directory
2. **Use meaningful target names** (not `temp1.csv`)
3. **Comment your Makefile** to explain steps
4. **Use variables** for paths and commands
5. **Add `.PHONY`** for non-file targets
6. **Test incrementally** as you build your Makefile

Common Pitfalls

✗ Using spaces instead of tabs

```
target: dependency
  command # WRONG! This is spaces
  command # RIGHT! This is a tab
```

✗ Circular dependencies

```
a.txt: b.txt
  cat b.txt > a.txt

b.txt: a.txt # CIRCULAR!
  cat a.txt > b.txt
```

Debugging Tips

Check what Make would do:

```
make -n target # Dry run, shows commands without executing
```

See why Make rebuilds:

```
make -d target # Debug mode, verbose output
```

Force rebuild:

```
make -B target # Rebuild regardless of timestamps
```

Example: Why Make > Shell Script

Shell script approach:

```
#!/bin/bash
Rscript download.R      # Always runs (even if data unchanged)
Rscript clean.R         # Always runs
Rscript analyze.R       # Always runs
Rscript plot.R          # Always runs
# Takes 30 minutes every time!
```

Makefile approach:

```
make all
# First run: 30 minutes
# Edit plot.R and re-run: 10 seconds (only reruns plotting!)
# No changes: instant (nothing to do!)
```

Hands-On Exercise (Optional)

Try this yourself:

1. Create a simple `Makefile`
2. Add a target that creates a text file
3. Add a target that depends on that file
4. Run `make` and observe behavior
5. Touch a file and run `make` again

Example starter:

```
hello.txt:  
    echo "Hello, Make!" > hello.txt  
  
goodbye.txt: hello.txt  
    cat hello.txt | sed 's/Hello/Goodbye/' > goodbye.txt
```

Advanced Topics (For Later)

Once comfortable with basics:

- **Automatic variables** (`$@`, `$<`, `^`)
- **Functions** (`wildcard`, `patsubst`)
- **Conditional execution** (`ifeq`, `ifdef`)
- **Including other Makefiles**
- **Integration with version control**
- **Make for reports** (Quarto, R Markdown)

Resources

Documentation:

- GNU Make Manual: <https://www.gnu.org/software/make/manual/>
- Make for reproducible research: <https://makefiletutorial.com/>

Tutorials:

- Software Carpentry: Make for Data Analysis
- Minimal Make by Karl Broman

Getting Help:

- `man make` (in terminal)
- Stack Overflow: [\[makefile\]](#) tag

Key Takeaways

- ✓ **Make tracks dependencies between files**
- ✓ **Only rebuilds what's necessary (saves time!)**
- ✓ **Documents your workflow in one place**
- ✓ **Works with any tools (R, Python, bash, etc.)**
- ✓ **Essential for reproducible research**

Your Next Steps

1. **Start small:** Convert one script to a Makefile
2. **Build incrementally:** Add one target at a time
3. **Test often:** Run `make` frequently
4. **Read the manual:** When you need advanced features
5. **Share your Makefiles:** Help others reproduce your work!

Questions? 🤔

Thank you for attending!

Remember: Reproducible research benefits everyone!

Makefile = Your research lab notebook 📖

Start using Make in your next project! 🚀

Optional: Installing GNU Make on Windows

Option 1: Using Chocolatey (Recommended)

1. Install Chocolatey (if not already installed):

- Right-click Start menu → “Windows PowerShell (Admin)”
- Visit: <https://chocolatey.org/install>
- Copy and run the installation command from the website

2. Install GNU Make:

```
choco install make
```

3. Verify installation:

```
make --version  
# Should show: GNU Make 4.x
```

Option 2: Using Git for Windows (Built-in)

- Download and install **Git for Windows**: <https://gitforwindows.org/>
- Make is included! Access via **Git Bash** terminal
- Look for “Git Bash” in your Start menu

Troubleshooting: - If `make` command not found → Restart terminal/PowerShell - Use Git Bash instead of Command Prompt for Option 2

Optional: Installing GNU Make on macOS

Option 1: Using Homebrew (Recommended)

1. Install Homebrew (if not already installed):

- Open **Terminal** (Applications → Utilities → Terminal)
- Visit: <https://brew.sh/>
- Copy and run the installation command:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install GNU Make:

```
brew install make
```

3. Verify installation:

```
make --version  
# Should show: GNU Make 4.x
```

Option 2: Xcode Command Line Tools (Built-in)

GNU Make comes with Xcode Command Line Tools: